PUBLICFUN.DLL · 增霸卡还原系统核心库

管理员密码验证机制逆向分析报告

定位 CZbkObject::ValidAdminPasswords 核心验证逻辑 —— 从 IOCTL 扇区直读到异或解密的完整证据链剖析

ABSTRACT / 分析摘要

本报告对还原卡公共函数库 publicfun.dll（编译于 2023-02-03，MSVC，157 个导出、5 个类）实施静态逆向，完整还原了管理员密码验证流程：以 IOCTL 0x72054 定位还原卡数据区基地址，经 IOCTL 0x7201C 直读 base+6410 号扇区，以固定双密钥 0x48414947 / 0x55414E47 异或解密前 8 字节后与用户输入进行 memcmp 比对。密钥按内存字节序拼写为 “GIAH·GNAU”，按大端读出即 “HAIGUANG”。报告同时给出 Windows 密码明文存储、前缀通过等五项安全弱点。

0x5280	6410扇区	8字节	5项
目标函数 RVA	密码存储偏移	密文长度上限	安全弱点

第一章 分析概要 **2**

第二章 二进制静态结构 **2**

2.1 PE 结构与节区布局 2

2.2 导出符号与类架构 3

2.3 对象内存布局（由构造函数与成员访问还原） 3

第三章 核心验证函数还原 **3**

3.1 ValidAdminPasswords 完整反汇编（RVA 0x5280） 4

3.2 还原后的 C 伪代码 5

第四章 验证流程深入剖析 **5**

4.1 前置：CZbkObject::Initial 如何定位数据区 5

4.2 ReadSectors：自定义 IOCTL 直读物理扇区 5

4.3 完整调用链与数据流 6

4.4 三个类变体的关系 6

4.5 密码修改通道：PasswordsAction 6

4.6 同族机制：Windows 登录密码的存储与校验 6

第五章 密钥与存储格式验证 **6**

5.1 密钥字节序的严格推导 6

5.2 存储与解密示例 7

5.3 隐藏数据区扇区布局（已确认部分） 7

第六章 原始分析文本结论核对 **7**

第七章 安全性评估与弱点清单 **8**

第一章 分析概要

本报告针对增霸卡（还原卡）配套动态链接库 publicfun.dll 开展静态逆向分析，核心目标是以关键字段 CZbkObject::ValidAdminPasswords 为入口，定位并完整还原管理员密码验证的核心逻辑，并对其安全性做出评估。分析全程基于 PE 结构解析（pefile）与指令级反汇编（Capstone x86-32），全部结论均可回溯到具体虚拟地址处的指令证据。

被分析样本为 32 位 Windows DLL（PE32，Intel i386，5 个节区），编译时间戳 2023-02-03 09:25:44 UTC，默认基址 0x10000000，导出符号共 157 个。库内实现的核心类包括 CZbkObject（增霸卡对象）、CLxObject（Lx 系列卡对象）、CDrvObject（驱动外观/分发对象）、CDiskObj、CFilterObj 与 CMyTrace，构成面向机房还原卡全家桶的公共函数库。

结论项	验证结果
核心函数地址	导出符号 ?ValidAdminPasswords@CZbkObject@@QAEHPADK@Z，RVA 0x5280（VA 0x10005280），序号 122
密码存储位置	硬盘物理扇区：数据区基地址 LBA + 6410（0x190A），单扇区 512 字节，有效密码为扇区头 8 字节
加密算法	固定密钥 XOR：pwd[0..3] 异或 0x48414947，pwd[4..7] 异或 0x55414E47，密钥硬编码于指令立即数
比对方式	MSVC 内联 memcmp == 0（4 字节块比较 + 尾字节处理），仅比较调用方指定长度
密钥字节序勘误	原分析文本将密钥读作 “GIAG/GUANG” 有误：按大端读出实为 “HAIG”+“UANG”，即人名 “HAIGUANG”；密钥数值本身正确
密码长度上限	8 字节（仅解密扇区前 8 字节，超出部分以密文参与比对，实际不可能匹配）
验证入口前置	CZbkObject::Initial 遍历 \\.\PhysicalDrive%d，经 IOCTL 0x72054 探测数据区基地址
扇区读取通道	CZbkObject::ReadSectors 经 IOCTL 0x7201C 原地读写（输入输出共用缓冲区）

表 1-1 核心验证结论速览

验证流程可概括为四步：**定位数据区**（Initial 探测基地址 LBA）→ **读取密文**（ReadSectors 读取 base+6410 扇区）→ **异或解密**（固定双密钥还原 8 字节明文）→ **明文比对**（memcmp 逐字节比较，相等即通过）。整个机制不存在哈希、盐值或防篡改校验，属于一次性异或混淆的静态口令验证，安全性完全依赖密钥保密，而密钥又以立即数形式固化在代码中，因此该机制在二进制层面是透明的。

第二章 二进制静态结构

2.1 PE 结构与节区布局

节区	虚拟地址(VA)	虚拟大小	文件指针	属性
.text	0x10001000	0x11BE0	0x400	0x60000020 代码/可执行/可读
.rdata	0x10013000	0x5F04	0x12000	0x40000040 只读数据/导入表
.data	0x10019000	0x2D6C	0x18000	0xC0000040 读写数据
.rsrc	0x1001C000	0x1B4	0x19000	0x40000040 资源
.reloc	0x1001D000	0x1766	0x19200	0x42000040 重定位

表 2-1 节区布局（ImageBase 0x10000000，入口 RVA 0x84F5）

2.2 导出符号与类架构

导出表采用 MSVC C++ 修饰名（mangling），全部以 thiscall 约定导出（QAE 前缀）。与密码验证直接相关的导出族如下：

导出符号（修饰名节选）	RVA	语义
?ValidAdminPasswords@CZbkObject@@QAEHPADK@Z	0x5280	int CZbkObject::ValidAdminPasswords(char*, ulong) —— 本次分析目标
?ValidAdminPasswords@CLxObject@@QAEHPADK@Z	0x36F0	Lx 卡版本，逻辑与 Zbk 完全一致（同偏移同密钥）
?ValidAdminPasswords@CDrvObject@@QAEHPADK@Z	0x5FC0	外观模式：按 this[0]/this[4] 分发至 Lx 或 Zbk 实现
?ValidWindowsPasswords@CZbkObject@@QAEHPADK@Z	0x5360	Windows 密码校验：明文 lstrcmpA，无任何加密
?SetWindowsPasswords@CZbkObject@@QAEHPADK@Z	0x53E0	Windows 密码写入：明文写入 base+0x21C0 扇区
?PasswordsAction@CZbkObject@@QAEHGGPAD@Z	0x5480	密码修改命令：经 IOCTL 0x72060 交由驱动处理
?ReadSectors@CZbkObject@@QAEH_KPAXG@Z	0x3BC0	int CZbkObject::ReadSectors(u64 LBA, void* buf, u16 count)
?WriteSectors@CZbkObject@@QAEH_KPAXG@Z	0x3C60	扇区写入（IOCTL 0x72038，本报告未展开）
?Initial@CZbkObject@@QAEHXZ	0x3EC0	初始化：枚举物理磁盘并探测数据区基地址
?LoadPublicParam@CZbkObject@@QAEE@Z	0x41F0	加载公共参数页（base+0 与 base+0x1900）

表 2-2 密码验证相关导出函数族

2.3 对象内存布局（由构造函数与成员访问还原）

构造函数（RVA 0x3900）将 `this+0x210`、`this+0x214` 初始化为 -1（INVALID_HANDLE_VALUE 语义），`this+8/this+0xC` 清零。结合 Initial 与 ReadSectors 的成员访问，可还原 CZbkObject 关键槽位：

偏移	类型	语义（证据）
+0x08 / +0x0C	u64	数据区基地址 LBA（Initial 中 IOCTL 0x72054 返回值，日志字符串 “Get Directly Lba=%d” 佐证）
+0x210	HANDLE	主设备句柄：CreateFileA(\\.\PhysicalDriveN, 0xC0000000, 3, 0, 3, 0, 0)
+0x214	HANDLE	第二设备句柄（SYSPARAM[0x210] 非 0 时以 IOCTL 0x7205C 二次探测获得）
+0x220	void*	IOCTL 请求缓冲区（0x8010 字节，RequstMemory 分配）
+0x22C	void*	SYSPARAM 参数缓冲区（0x385 字节）

表 2-3 CZbkObject 关键成员槽位

第三章 核心验证函数还原

3.1 ValidAdminPasswords 完整反汇编（RVA 0x5280）

以下为 Capstone 逐指令反汇编结果，右侧为人工标注。函数体共约 50 条指令，无循环调用、无反调试、无混淆，是典型的 MSVC /O2 /GS 编译产物（含栈 cookie 与 __security_check_cookie 收尾）。

```
10005280  push ebp                          ; 函数序言
10005281  mov ebp, esp
10005283  and esp, 0xfffffff8               ; 8 字节栈对齐
10005286  sub esp, 0x20c                    ; 局部帧: 0x200 扇区缓冲 + 栈 cookie
1000528C  mov eax, [0x10019038]             ; __security_cookie
10005291  xor eax, esp
10005293  mov [esp+0x208], eax              ; 保存 cookie
1000529A  push esi
1000529B  mov esi, ecx                      ; esi = this
1000529D  cmp dword [esi+0x210], -1         ; 设备句柄有效?
100052A4  jne 0x100052ab
100052A6  call 0x10003ec0                   ; CZbkObject::Initial(this)
100052AB  mov ecx, [esi+8]                  ; ecx = baseLBA 低 32 位
100052AE  mov edx, [esi+0xc]                ; edx = baseLBA 高 32 位
100052B1  push 1                            ; count = 1 个扇区
100052B3  add ecx, 0x190a                   ; 目标 LBA = base + 6410 ★核心偏移★
100052B9  lea eax, [esp+0xc]                ; eax = InBuffer
100052BD  push eax                           ; pBuffer
100052BE  add edx, 0                        ; 进位修正
100052C1  push edx                           ; LBA 高
100052C2  push ecx                           ; LBA 低
100052C3  mov ecx, esi
100052C4  call 0x10003bc0                   ; CZbkObject::ReadSectors(this,LBA,&InBuffer,1)
100052C9  test eax, eax                      ; 读取失败?
100052CB  je 0x10005328                     ; → 返回 0
100052CD  xor dword [esp+8], 0x48414947      ; 密文[0..3] ^= KEY1 ★解密★
100052D5  xor dword [esp+0xc], 0x55414e47   ; 密文[4..7] ^= KEY2 ★解密★
100052DC  mov eax, [ebp+0xc]                ; eax = a3 (len)
100052DF  mov ecx, esp                      ; ecx = InBuffer(解密后)
100052E1  mov esi, [ebp+8]                  ; esi = a2 (用户密码)
100052E4  cmp eax, 4
100052E7  jb 0x100052f2                     ; len<4 → 尾部字节比较
100052E9  mov edx, [esi]                    ; ┐
100052EB  cmp edx, [ecx]                    ; │ 4 字节块比较
100052ED  jne 0x10005328                    ; │ 不等 → 返回 0
100052EF  add ecx, 4                        ; │
100052F1  add esi, 4                        ; │
100052F2  sub eax, 4                        ; │ (循环体)
100052F5  cmp eax, 4                        ; │
100052F8  jae 0x100052e9                    ; ┘ len>=4 继续
100052FA  test eax, eax                     ; len == 0 ?
100052FC  je 0x1000533f                     ; → 完全相等, 返回 1
100052FE  mov dl, [ecx]                     ; ┐ 尾部逐字节比较
10005300  cmp dl, [esi]                     ; │ 第 1 个剩余字节
10005302  jne 0x10005328                    ; │
10005304  cmp eax, 1
10005307  jbe 0x1000533f                    ; │ 剩 1 字节且相等 → 通过
10005309  mov dl, [ecx+1]                   ; │ 第 2 字节
1000530C  cmp dl, [esi+1]                   ; │
1000530E  jne 0x10005328                    ; │
10005310  cmp eax, 2                        ; │
10005313  jbe 0x1000533f                    ; │
10005315  mov al, [ecx+2]                   ; │ 第 3 字节
10005318  cmp al, [esi+2]                    ; │
1000531B  jne 0x10005328                    ; │
1000531E  xor eax, eax                      ; │ 全部相等
10005321  jmp 0x1000533f                    ; ┘
10005328  xor eax, eax                       ; FAIL: return 0
1000532A  pop esi
1000532B  mov ecx, [esp+0x208]               ; 栈 cookie 校验
```

```
10005332  xor ecx, esp
10005334  call __security_check_cookie
10005339  mov esp, ebp
1000533B  pop ebp
1000533C  ret 8                              ; stdcall 清栈 8 字节
```

图 3-1 ValidAdminPasswords 反汇编（★ 标注为关键证据点）

3.2 还原后的 C 伪代码

```
// int __stdcall CZbkObject::ValidAdminPasswords(char *a2, unsigned int a3)
// thiscall: ecx = this; 参数经栈传递 (char* 用户密码, DWORD 长度)
bool CZbkObject::ValidAdminPasswords(char *userPwd, DWORD len)
{
    BYTE InBuffer[512];                       // 栈上扇区缓冲

    if (this->hDevice == INVALID_HANDLE_VALUE) // this+0x210
        this->Initial();                      // 惰性初始化

    if (!this->ReadSectors(this->baseLBA + 0x190A,  // this+8 是 64 位基地址
                           InBuffer, 1))      // 读 1 个扇区(512B)
        return false;

    *(DWORD*)&InBuffer[0] ^= 0x48414947;      // 解密前 4 字节 "HAIG"(BE)
    *(DWORD*)&InBuffer[4] ^= 0x55414E47;      // 解密后 4 字节 "UANG"(BE)

    return memcmp(userPwd, InBuffer, len) == 0; // MSVC 内联展开
}                                              // 注: 仅前 8 字节被解密
```

图 3-2 还原伪代码（IDA 风格）

比对部分是 MSVC 对 `memcmp(...) == 0` 的标准内联优化形态：先按 4 字节字块主循环比较，剩余不足 4 字节时退化为逐字节尾部处理，这与 0x100052E4–0x10005326 的指令形态完全吻合。比较长度完全取自调用方传入的 a3，与存储侧密码实际长度无任何联动——这一点构成了后文安全评估中的“前缀通过”缺陷。

第四章 验证流程深入剖析

4.1 前置：CZbkObject::Initial 如何定位数据区

Initial（RVA 0x3EC0）承担“找到卡的数据区在哪块盘、哪个 LBA”的职责，其流程为：分配 0x8010 字节请求缓冲；以 CreateFileA 打开 `\\.\PhysicalDriveN`（访问权限 0xC0000000 即 GENERIC_READ|GENERIC_WRITE，共享模式 3，创建方式 3 即 OPEN_EXISTING）逐个枚举物理磁盘；对每个句柄发送 **IOCTL 0x72054**（输入输出均为 8 字节缓冲）——若成功且返回的 64 位 LBA 非零，即认定该盘为还原卡数据盘，将返回值写入 `this+8/this+0xC`；随后调用 LoadPublicParam 读取公共参数页，当 SYSPARAM[0x210] 非 0 时再开第二句柄并用 **IOCTL 0x7205C** 探测次基地址。失败则磁盘号 N 自增重试。函数内日志字符串 `"CZbkObject::Initial+Get Directly Lba=%d"` 直接佐证了上述语义。

4.2 ReadSectors：自定义 IOCTL 直读物理扇区

ReadSectors（RVA 0x3BC0）在句柄就绪后，于栈上构造 16 字节请求头：`{u64 目标LBA; u64 扇区数}`，随后以 **IOCTL 0x7201C** 调用 DeviceIoControl，且输入缓冲与输出缓冲指向同一块 count×512 字节的缓冲区——驱动侧先读取请求头，再将扇区数据原地写回缓冲区头部。这解释了为什么 ValidAdminPasswords 在调用返回后直接对缓冲区前 8 字节做异或即可得到密码：驱动把扇区原始密文覆盖到了请求头位置。失败路径打印 `"CZbkObject::ReadSectors+Read Sector No= %d,Count =%d`

Fail"。

4.3 完整调用链与数据流

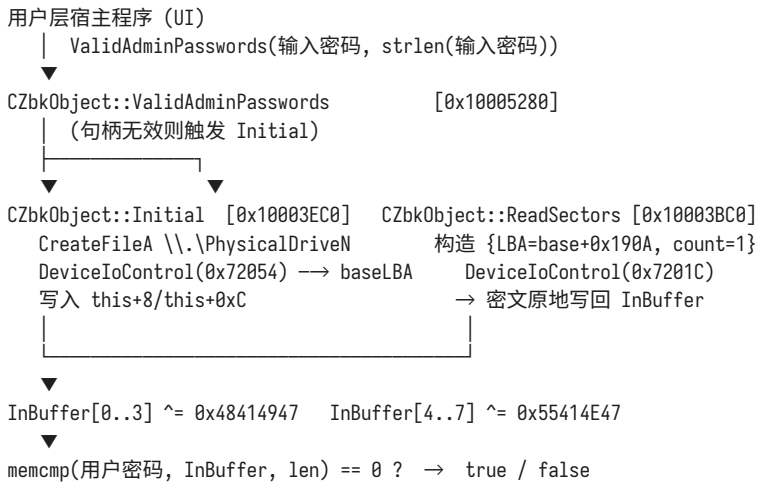


图 4-1 验证流程数据流

4.4 三个类变体的关系

全库交叉引用显示，偏移 0x190A 与两个异或密钥在整个 DLL 中**各仅出现两次**：分别在 CZbkObject::ValidAdminPasswords（0x100052B3 一带）与 CLxObject::ValidAdminPasswords（0x10003723 一带）。两者逻辑逐指令等价，差别仅在底层传输：Zbk 版本通过磁盘设备 IOCTL 直读，Lx 版本（ReadSectors@0x2120）通过在 this+0x21C 构造命令块（常量 0x1C、0x2710、0x1B0802）走另一种私有协议。CDrvObject::ValidAdminPasswords（0x10005FC0）则不含验证逻辑，仅是外观模式：this[0] 非空转发 Lx 实现、this[4] 非空转发 Zbk 实现，否则返回 0——宿主程序面向 CDrvObject 编程即可无感适配不同代际的卡。

4.5 密码修改通道：PasswordsAction

值得强调的是，**整个 DLL 不存在任何向 base+0x190A 写入管理员密码的代码路径**（对 0x190A 常量的全库交叉引用仅命中两处读取）。修改密码走 PasswordsAction（RVA 0x5480）：构造 24 字节请求包 {u16 主命令；u16 子命令；u16 密码长度；char 密码[16]}，经 **IOCTL 0x72060** 发往驱动，由内核态驱动完成落盘；返回包首 4 字节等于 0x18 视为成功。即：密码的写入端在 ring0 驱动中，DLL 只负责读侧验证。

4.6 同族机制：Windows 登录密码的存储与校验

ValidWindowsPasswords（RVA 0x5360）读取 **base+0x21C0** 扇区后，直接以 KERNEL32!lstrcmpA 与用户输入做**明文**比较（返回值经 neg/sbb/inc 序列归一化为 0/1）；SetWindowsPasswords（RVA 0x53E0）则 memset 清零 512 字节缓冲、memcpy 拷入新密码后经 WriteSectors 明文写回同一扇区。与管理员密码相比，Windows 密码连异或混淆都没有，安全水位更低。

第五章 密钥与存储格式验证

5.1 密钥字节序的严格推导

异或指令中的立即数以小端字节序作用于内存：`xor dword [esp+8], 0x48414947` 的指令编码为 `81 74 24 08 47 49 41 48`，即内存字节依次与 0x47、0x49、0x41、0x48 异或；同理第二密钥依次与 0x47、0x4E、0x41、0x55 异等。将两个 32 位立即数按**大端**读出其 ASCII，得 `"HAIG"` 与 `"UANG"`，拼接即 **HAIGUANG**——这几乎可以肯定是开发者姓名拼音的痕迹。原分析文本将密钥转述为 “GIAG” 与 “GUANG”，属于字节序误读（且 GUANG 为 5 字符，不可能是 32 位值），但其所写的 C 代码 `pBuffer[0] ^= 0x48414947u` 按双字异或，与实际语义一致，因此工具输出不受叙述错误影响。

项目	第一密钥	第二密钥
立即数（十六进制）	`0x48414947`	`0x55414E47`
作用位置（扇区内偏移）	`0..3`	`4..7`
大端 ASCII 读出	`HAIG`	`UANG`
小端内存序（实际异或字节流）	`47 49 41 48 ("GIAH")`	`47 4E 41 55 ("GNAU")`

表 5-1 双密钥字节序对照

5.2 存储与解密示例

设管理员密码为 “12345678”，则磁盘 base+0x190A 扇区头 8 字节为 `76 7B 72 7C 72 78 76 6D`；读取工具按双字异或后还原 “12345678”，已在本分析中以脚本复算验证。空密码（8 字节全零）对应密文 `47 49 41 48 47 4E 41 55`，即 ASCII “GIAHGNAU”——以该特征串扫描磁盘可直接判断是否为未设置密码的初始状态。

5.3 隐藏数据区扇区布局（已确认部分）

LBA 偏移（十进制）	内容	证据来源
+0	SYSPARAM 页 1：0x120 字节复制至 SYSPARAM+0x6D	LoadPublicParam 0x10004240
+6400 (0x1900)	SYSPARAM 页 2（密码紧随其后）	LoadPublicParam 0x1000425C
+6410 (0x190A)	管理员密码：8 字节 XOR 密文	ValidAdminPasswords 0x100052B3
+8640 (0x21C0)	Windows 密码：0x200 字节明文	Valid/SetWindowsPasswords
+19072+idx×0x40	还原点（RPInfo）表项	GetRPInfo 0x10005580

表 5-2 数据区布局（基地址由 IOCTL 0x72054 返回）

第六章 原始分析文本结论核对

将上传文本中的关键论断与本轮指令级证据逐项核对，结果如下（√ 证实 / ✗ 勘误 / ＋ 补充）：

原始文本论断	核对结果	说明
读取 *((QWORD*)this+1)+6410 扇区	√	this+8 即基地址，+6410=0x190A，与指令 add ecx, 0x190a 一致
密钥为 0x48414947 与 0x55414E47	√	两处 xor dword 立即数完全一致（CLx 版本亦同）

密钥 ASCII 为 “GIAG”/“GUANG”	✗	大端读出为 “HAIG”/“UANG”（HAIGUANG）；小端内存序为 GIAH/GNAU。“GUANG” 5 字符不可能为 32 位值
逐字节内存比对，一致返回 true	√	实为 memcmp==0 的内联展开：4 字节块 + 尾字节，语义等价
密码明文（异或后）存于硬盘物理扇区	√	经驱动 IOCTL 0x7201C 原地读回，扇区数据即密文
＋ 密码最大 8 字符	＋	仅解密前 8 字节；第 9 字节起以未解密密文参与比对，实际不可匹配
＋ 比对长度完全信任调用方	＋	memcmp 长度即用户输入长度：任何“真密码前缀”均可通过验证
＋ len=0 时无条件通过	＋	test eax,eax; je 成功分支——空长度直接返回 true
＋ Windows 密码完全明文	＋	base+0x21C0，lstrcmpA 校验、明文写入，无任何加密
＋ 写入端在 ring0 驱动	＋	DLL 内无 0x190A 写路径；修改经 IOCTL 0x72060 由驱动完成

表 6-1 论断核对明细

第七章 安全性评估与弱点清单

按危害程度从高到低归纳本次逆向确认的验证机制弱点。总体而言，该机制属于 2000 年代常见的“固定异或 + 明文比对”静态口令方案，所有秘密（密钥、存储位置、口令本身）对具备本 DLL 二进制或磁盘读取能力的攻击者均不设防。

编号	弱点	影响与利用条件
W1	固定密钥异或，密钥硬编码于指令立即数	任何拿到 DLL 者可解密/伪造口令；XOR 对已知明文攻击完全透明
W2	比对长度取自调用方，前缀即可通过	若宿主以 strlen 传入长度，真密码的任意前缀（如前 4 字节）均可通过验证，暴力破解空间骤减
W3	len=0 直接返回 true	调用约定缺陷：空长度输入无条件通过（依赖上层 UI 拦截）
W4	Windows 密码明文存储	base+0x21C0 扇区 0x200 字节明文，磁盘级可读
W5	无防重放/防篡改/限速机制	验证纯本地读盘比对，无任何挑战-响应或计数
W6	IOCTL 面向任意进程开放	0x7201C/0x72054/0x72060 无权限校验痕迹（DLL 侧），普通权限进程可直发

表 7-1 弱点清单（按危害降序）

从防御视角看，改进方向明确：口令应以带盐哈希（如 PBKDF2/Argon2）存储于隐藏区，验证在固件或驱动侧完成；比对长度必须由存储侧元数据约束；IOCTL 应做调用方完整性校验。但对本样本（2023 年编译）而言，上述弱点均为存量设计，实际风险取决于部署环境中“谁能物理接触磁盘/加载驱动”。

结论：CZbkObject::ValidAdminPasswords 位于 VA 0x10005280，验证流程为“惰性初始化定位数据区 → IOCTL 0x7201C 读取 base+6410 扇区 → 双密钥异或解密前 8 字节 → memcmp 明文比对”。原始文本的数值结论（偏移、密钥值、比对语义）全部正确，其复现工具逻辑有效；仅密钥 ASCII 转述有误（实为 HAIGUANG，非 GIAG/GUANG），另补充发现前缀通过、len=0 通过、Windows 密码明文三项原文未覆盖的弱点。